

Advanced Data Analytics

- 1) Analyze monthly sales performance to identify growth patterns, seasonality, and customer purchasing behavior over time.

SELECT

```
YEAR(order_date) AS order_year,  
MONTH(order_date) AS order_month,  
SUM(sales_amount) AS total_sales,  
COUNT(DISTINCT customer_key) AS total_customers,  
SUM(quantity) AS total_quantity
```

FROM gold.fact_sales

WHERE order_date IS NOT NULL

GROUP BY YEAR(order_date), MONTH(order_date)

ORDER BY order_year, order_month;

	order_year	order_month	total_sales	total_customers	total_quantity
1	2010	12	43419	14	14
2	2011	1	469795	144	144
3	2011	2	466307	144	144
4	2011	3	485165	150	150
5	2011	4	502042	157	157
6	2011	5	561647	174	174
7	2011	6	737793	230	230
8	2011	7	596710	188	188
9	2011	8	614516	193	193
10	2011	9	603047	185	185
11	2011	10	708164	221	221
12	2011	11	660507	208	208
13	2011	12	669395	222	222
14	2012	1	495363	252	252
15	2012	2	506992	260	260
16	2012	3	373478	212	212
17	2012	4	400224	210	210

- 2). Running total and moving average analysis to Track sales accumulation over time and smooth price trends using averages.

SELECT

```
order_date,  
total_sales,
```

-- Cumulative sales to measure business growth over time

```

SUM(total_sales) OVER (ORDER BY order_date) AS running_total_sales,

-- Moving average to observe pricing stability trends
AVG(avg_price) OVER (ORDER BY order_date) AS moving_avg_price

FROM (
  SELECT
    DATETRUNC(year, order_date) AS order_date,
    SUM(sales_amount) AS total_sales,
    AVG(sls_price) AS avg_price
  FROM gold.fact_sales
  WHERE order_date IS NOT NULL
  GROUP BY DATETRUNC(year, order_date)
) t;

```

127 % No issues found

Results Messages

	order_date	total_sales	running_total_sales	moving_avg_price
1	2010-01-01	43419	43419	3101
2	2011-01-01	7075088	7118507	3146
3	2012-01-01	5842231	12960738	2670
4	2013-01-01	16344878	29305616	2080
5	2014-01-01	45642	29351258	1668

Query executed successfully.

3) Yearly Product Performance Analysis: -- Evaluate how each product performs over time by comparing yearly sales against historical averages and identifying growth trends.

```

WITH yearly_product_sales AS (
    SELECT
        YEAR(f.order_date) AS order_year,
        p.product_name,
        SUM(f.sales_amount) AS current_sales
    FROM gold.fact_sales f
    LEFT JOIN gold.dim_products p
        ON f.product_key = p.product_key
    WHERE f.order_date IS NOT NULL
    GROUP BY YEAR(f.order_date), p.product_name
)

SELECT
    order_year,
    product_name,
    current_sales,

    -- Average performance of product across years
    AVG(current_sales) OVER (PARTITION BY product_name) AS avg_sales,

    -- Difference from historical average
    current_sales - AVG(current_sales) OVER (PARTITION BY product_name) AS diff_from_avg,

    -- Performance classification
    CASE
        WHEN current_sales > AVG(current_sales) OVER (PARTITION BY product_name) THEN 'Above Average'
        WHEN current_sales < AVG(current_sales) OVER (PARTITION BY product_name) THEN 'Below Average'
        ELSE 'Stable'
    END AS performance_status,

    -- Previous year comparison
    LAG(current_sales) OVER (
        PARTITION BY product_name
        ORDER BY order_year
    ) AS previous_year_sales,

    -- Year-over-year change
    current_sales - LAG(current_sales) OVER (
        PARTITION BY product_name
        ORDER BY order_year
    ) AS yoy_difference,

    -- Growth direction
    CASE
        WHEN current_sales > LAG(current_sales) OVER (
            PARTITION BY product_name ORDER BY order_year
        ) THEN 'Increase'
        WHEN current_sales < LAG(current_sales) OVER (
            PARTITION BY product_name ORDER BY order_year
        ) THEN 'Decrease'
        ELSE 'No Change'
    END AS yoy_trend

```

```
FROM yearly_product_sales
ORDER BY product_name, order_year;
```

	order_year	product_name	current_sales	avg_sales	diff_from_avg	performance_status	previous_year_sales	yoy_difference	yoy_trend
1	2012	All-Purpose Bike Stand	159	13197	-13038	Below Average	NULL	NULL	No Change
2	2013	All-Purpose Bike Stand	37683	13197	24486	Above Average	159	37524	Increase
3	2014	All-Purpose Bike Stand	1749	13197	-11448	Below Average	37683	-35934	Decrease
4	2012	AWC Logo Cap	72	6570	-6498	Below Average	NULL	NULL	No Change
5	2013	AWC Logo Cap	18891	6570	12321	Above Average	72	18819	Increase
6	2014	AWC Logo Cap	747	6570	-5823	Below Average	18891	-18144	Decrease
7	2013	Bike Wash - Dissolver	6960	3636	3324	Above Average	NULL	NULL	No Change
8	2014	Bike Wash - Dissolver	312	3636	-3324	Below Average	6960	-6648	Decrease
9	2013	Classic Vest- L	11968	6240	5728	Above Average	NULL	NULL	No Change
10	2014	Classic Vest- L	512	6240	-5728	Below Average	11968	-11456	Decrease
11	2013	Classic Vest- M	11840	6368	5472	Above Average	NULL	NULL	No Change
12	2014	Classic Vest- M	896	6368	-5472	Below Average	11840	-10944	Decrease
13	2012	Classic Vest- S	64	3648	-3584	Below Average	NULL	NULL	No Change
14	2013	Classic Vest- S	10368	3648	6720	Above Average	64	10304	Increase
15	2014	Classic Vest- S	512	3648	-3136	Below Average	10368	-9856	Decrease
16	2012	Fender Set - Mountain	110	15554	-15444	Below Average	NULL	NULL	No Change
17	2013	Fender Set - Mountain	44494	15554	28940	Above Average	110	44374	Increase

-- 4. CATEGORY CONTRIBUTION TO TOTAL SALES: -- Understand how each product category contributes to total revenue.

```
WITH category_sales AS (
  SELECT
    p.category,
    SUM(f.sales_amount) AS total_sales
  FROM gold.fact_sales f
  LEFT JOIN gold.dim_products p
    ON f.product_key = p.product_key
  GROUP BY p.category
)

SELECT
  category,
  total_sales,

  -- Total revenue across all categories
  SUM(total_sales) OVER () AS overall_sales,

  -- Percentage contribution of each category
  CONCAT(
    ROUND(total_sales * 100.0 / SUM(total_sales) OVER (), 2),
    '%'
  ) AS sales_contribution
FROM category_sales
ORDER BY total_sales DESC;
```

95 %   No issues found

 Results

 Messages

	category	total_sales	overall_sales	sales_contribution
1	Bikes	28316272	29356250	96.4600000000000%
2	Accessories	700262	29356250	2.3900000000000%
3	Clothing	339716	29356250	1.1600000000000%

Query executed successfully.

-- 5. PRODUCT SEGMENTATION BY COST RANGE:-- Group products into pricing segments to understand product distribution.

```

WITH product_segments AS (
  SELECT
    product_key,
    product_name,
    cost,

    CASE
      WHEN cost < 100 THEN 'Below 100'
      WHEN cost BETWEEN 100 AND 500 THEN '100-500'
      WHEN cost BETWEEN 500 AND 1000 THEN '500-1000'
      ELSE 'Above 1000'
    END AS cost_segment
  FROM gold.dim_products
)

SELECT
  cost_segment,
  COUNT(product_key) AS total_products
FROM product_segments
GROUP BY cost_segment
ORDER BY total_products DESC;

```

95 %



No issues found



Results



Messages

	cost_segment	total_products
1	Below 100	110
2	100-500	101
3	500-1000	45
4	Above 1000	39



Query executed successfully.